PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2

2da práctica (tipo b)
Segundo Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales, estructuras, NI OBJETOS** (con excepción de los elementos de **iostream, iomanip y fstream**). **NO PUEDE UTILIZAR LA CLASE string**. Tampoco se podrán emplear las funciones malloc, realloc, memset, strtok o strdup, igualmente no se puede emplear cualquier función contenida en las bibliotecas stdio.h, cstdio o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ESTRICTO DISEÑO DESCENDENTE. **Cada función NO debe sobrepasar las 20 líneas de código aproximadamente.** El archivo main.cpp solo podrá contener la función main de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo main.cpp deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final.**
- El código comentado NO SE CALIFICARÁ. De igual manera NO SE CALIFICARÁ el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestres resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.

SE LES RECUERDA QUE, DE ACUERDO CON EL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- **Puntaje total**: 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLION siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"CO_PA_PN_Lab02_2026_2"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará los proyectos solicitados en la prueba.**
- En cada proyecto que desarrolle deberá crear una carpeta denominada "**Bibliotecas**" y allí coloque la biblioteca de funciones que necesite. Dentro de la carpeta "**cmake-build-debug**" cree las carpetas "**ArchivosDeDatos**" y "**ArchivosDeReporte**" y allí coloque respectivamente el archivo de datos que se le proporcionará y el archivo del reporte solicitado en la prueba. De no colocar esto se descontará un punto en cada proyecto.

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 2 del curso: "Arreglos y punteros". En este laboratorio se trabajará con memoria dinámica y los métodos de asignación exacta de memoria y por incrementos.

Deberá elaborar un proyecto denominado "AsignacionEficienteDeMemoria-2026-2" y en él desarrollará el programa que dé solución al problema planteado. DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL. **NO SE HARÁN EXCEPCIONES**

NO PODRÁ EMPLEAR ARREGLOS ESTÁTICOS DE MÁS DE UNA DIMENSIÓN
NO PUEDE MANIPULAR UN PUNTERO CON MÁS DE UN ÍNDICE
NO PUEDE EMPLEAR ARREGLOS AUXILIARES, ESTÁTICOS O DINÁMICOS, PARA GUARDAR LOS DATOS DEL ARCHIVO
LOS ARCHIVOS SOLO SE PUEDEN LEER UNA VEZ

PackMart es una empresa ficticia dedicada a la venta minorista y mayorista de productos de consumo. La empresa administra productos que pueden venderse por unidad base o mediante presentaciones comerciales, por ejemplo, paquetes de 10, 20, 25 o 50 unidades y cajas de 12, 24, 48 o 100 unidades. Entre otros, la empresa maneja los siguientes archivos.

Productos.csv

```
P0001,Avena Pacifico Tipo 17,ABARROTES,22,25
P0002,Bebida Isotonica Costa Tipo 24,BEBIDAS,40,56
…
```

Cada línea del archivo representa un producto comercializado por la empresa y contiene, en el siguiente orden: código del producto, descripción, categoría, costo unitario y precio unitario de venta.

Ventas.csv

```
V00001,C0277,2026-07-07,WEB
V00002,C0407,2026-06-09,TIENDA
…
```

Cada línea del archivo representa una venta realizada por la empresa y contiene, en el siguiente orden: código de la venta, código del cliente, fecha de la venta en formato YYYY-MM-DD y canal de venta (WEB, TIENDA, etc.).

DetalleVentas.csv

```
V00001,P0149,UNIDAD,1,4,0%
V00002,P0258,CAJA12,12,8,13%
V00001,P0417,PAQ10,10,5,8%
…
```

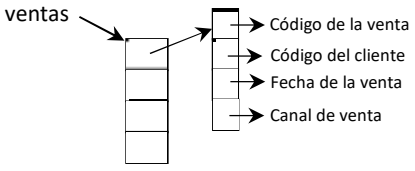
Cada línea del archivo representa un producto incluido en una venta y contiene, en el siguiente orden: código de la venta, código del producto, presentación solicitada, número de unidades por presentación, cantidad comprada y porcentaje de descuento aplicado.

Por ejemplo, en CAJA12,12,8, CAJA12 indica la presentación solicitada, 12 indica que cada caja contiene 12 unidades del producto y 8 indica que se compraron 8 cajas.

Una venta puede contener varios productos; por ello, un mismo código de venta puede aparecer en múltiples líneas del archivo. Estas líneas no necesariamente se encuentran almacenadas de manera consecutiva.

Con la información dada se deberá elaborar un proyecto con CLion en C++ que permita realizar las siguientes tareas:

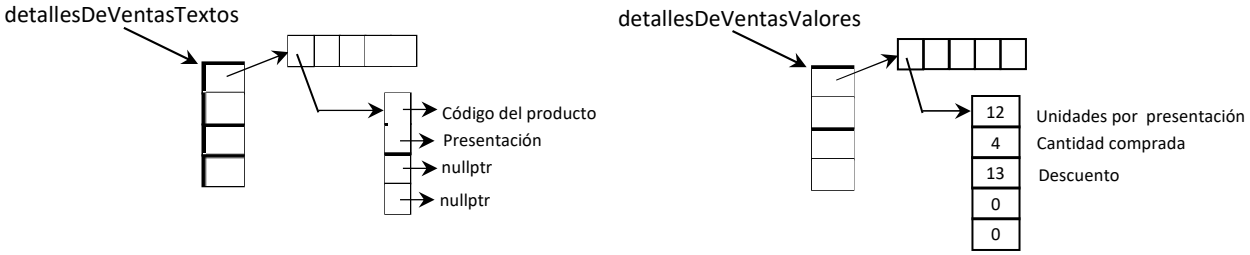
1. (4 puntos) Implementa la tarea **cargarVentas** que recibirá dos argumentos: el nombre del archivo de ventas y un puntero que finalmente apuntará a un arreglo dinámico que contenga todos los datos del archivo como la que se muestra en la figura siguiente:



   Todos los datos serán manejados como cadenas de caracteres, debe manejar el método de asignación de memoria dinámica exacta en todos los niveles.

   Implemente, además, una función independiente que permita verificar la correcta carga de los dato sen un archivo de reporte. El reporte debe tener un título que indique de qué datos se trata, encabezados simples encima de cada columna que indique lo que contienen y los datos con un formato y alineación apropiado de acuerdo con el tipo de dato. La función debe recibir el nombre del reporte y este será "ReporteVentas.txt".

2. (6 puntos) Implemente la tarea **cargarDetallesDeVentas** que recibirá cuatro argumentos: el nombre del archivo con los detalles de ventas, un puntero al arreglo "ventas" creada en la tarea anterior y dos punteros que finalmente apuntará a arreglos dinámicos que contengan todos los datos del archivo como la que se muestra en la figura siguiente:



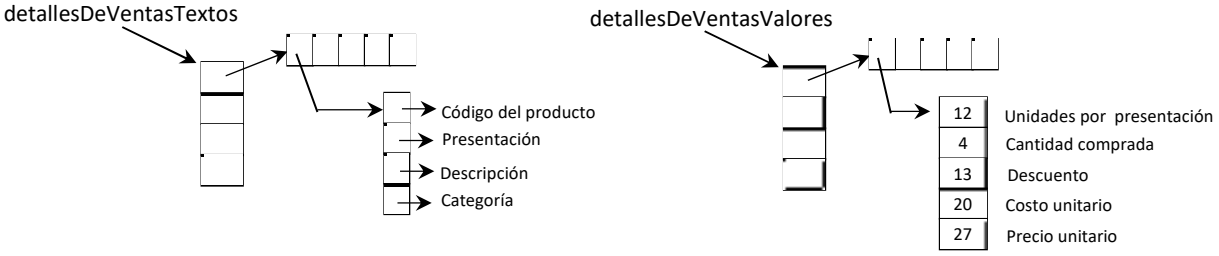
Los elementos de estos arreglos deberán corresponder con los elementos almacenados en el arreglo de ventas creado en la tarea anterior. Para cada venta se deberán almacenar todos los productos asociados a ella.

El archivo **DetalleVentas.csv** solo se podrá leer una vez.

Para los arreglos secundarios que almacenan los detalles de cada venta deberá emplearse **asignación dinámica por incrementos**, el incremento se fijará en 5. Los otros elementos se deben asignar **de manera exacta**.

Modifique la función de verificación anterior de modo que pueda imprimir el reporte anterior pero también los datos de esta tarea. Los cambios deben hacerse de modo tal que no tenga que borrar ni comentar la instrucciones prueba de la tarea anterior. La función debe recibir el nombre del reporte y este será **"ReporteVentasConDetalles.txt"**.

3. (6 puntos) Implemente la tarea **completarDetallesDeVentas** y recibirá cuatro argumentos: el nombre del archivo con los productos de la empresa, un puntero al arreglo "ventas" y los punteros "**detallesDeVentasTextos**" y "**detallesDeVentasValores**". Utilizando la información contenida en **Productos.csv**, complete los detalles de cada producto vendido incorporando su **descripción**, **categoría**, **costo unitario** y **precio unitario**, de acuerdo con la organización mostrada en la siguiente figura.



El archivo de **Productos.csv** solo puede leerse una vez.

Modifique la función de verificación anterior de modo que también pueda imprimir los reportes. Los cambios deben hacerse de modo tal que no tenga que borrar ni comentar las instrucciones de las pruebas de las tareas anteriores. La función debe recibir el nombre del reporte y este será "**ReporteVentasConDetallesProductos.txt**".

4. (4 puntos) Modifique la función de verificación anterior de modo que presente un reporte como el que se muestra a continuación. Los cambios deben hacerse de modo tal que no tenga que borrar ni comentar las pruebas de las tareas anteriores. La función debe recibir el nombre del reporte y este será "ReporteVentasCompleto.txt". **El reporte debe incluir para cada venta, el cálculo del costo total de los productos, el precio total de venta y el monto de la ganancia por la venta.**

PackMart S.A.
REGISTRO DE VENTAS

CODIGO DE VENTA	CODIGO DEL CLIENTE	FECHA DE VENTA	CANAL DE VENTA
V00001	C0277	2026-07-07	WEB

PRODUCTOS COMPRADOS:

CODIGO	DESCRIPCION	PRESENTACION	CATEGORIA	UNID X PRES	CANTIDAD	COTSTO UNIT	PRECIO UNIT	DESCUENTO
P0106	Te Inti Tipo 11	PAQ20	BEBIDAS	20	5	9	14	13%
…	…	…	…	…	…	…	…	…

COSTO TOTAL: 9764.00
PRECIO TOTAL: 12799.33
GANANCIA TOTAL: 3035.33

CODIGO DE VENTA	CODIGO DEL CLIENTE	FECHA DE VENTA	CANAL DE VENTA
…	…	…	…

Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

Profesores del curso: Rony Cueva, Erasmo Gómez, Ana Roncal, Miguel Guanira

San Miguel, 11 de septiembre del 2026.